

Đại học Bách Khoa Hà Nội
TRƯỜNG ĐIỆN – ĐIỆN TỬ



Mã dự án: 25236

Tối ưu hóa Pipe để Cải thiện Thông lượng IPC trong xv6

Sinh viên:	Nguyễn Sỹ Tuấn	20224191
	Lê Huy Sơn	
	Vũ Tiến Long	
Lớp	169227	
Giảng viên hướng dẫn:		
Học phần	Hệ điều hành	

Hoàn thành: 21/05/2026

Mục lục

Phân công công việc & Đóng góp	3
1 Giới thiệu	4
1.1 Đặt vấn đề	4
1.2 Câu hỏi nghiên cứu	4
1.3 Cấu trúc báo cáo	4
2 Cơ sở lý thuyết	5
2.1 Inter-Process Communication (IPC)	5
2.2 Pipe trong UNIX/xv6	6
2.3 Circular Buffer (Bộ đệm vòng)	8
2.4 Cơ chế Spinlock và Sleep/Wakeup trong xv6	9
2.5 Cách pipe hoạt động với tiến trình	11
2.6 Thông lượng IPC (IPC Throughput)	12
3 Phân tích Baseline (v1)	12
3.1 Đường đi của một lần write/read qua pipe	12
3.2 Source code đầy đủ – pipealloc, pipewrite và piperead v1	14
3.2.1 struct pipe và pipealloc() v1	14
3.2.2 pipewrite() và piperead() v1	15
3.2.3 pipeclose() v1	17
3.3 Các phiên bản tối ưu thông lượng	19
3.4 Đo bằng thông của phiên bản v1	20
4 Các phiên bản tối ưu (v2–v8)	20
4.1 v2 – Bulk Copy	20
4.1.1 Kết quả đo đặc v2	23
4.2 v3 – Ring of Buffers	24
4.2.1 Giải pháp – Chunked 2D Ring Buffer	25
4.2.2 Kết quả đo đặc v3	26
4.3 v4 – Multi-page Allocation	26
4.3.1 Kết quả đo đặc v4	28
4.4 v5 – Lazy Wakeup	28
4.4.1 Phân tích đối xứng: trạng thái writer khi reader đang đọc	30
4.4.2 Kết quả đo đặc v5	31
4.5 v6 – Priority Boost	31
4.6 v7 – Cache-line Alignment	35
4.6.1 Nhận xét về kết quả v7 trên single-CPU	37

4.7	v8 – All Combined (Tổng hợp tất cả cơ chế v2–v7)	37
4.7.1	Mục tiêu – Kết hợp 5 cơ chế thành 1 phiên bản hoàn chỉnh	37
4.7.2	Thiết kế v8	37
4.7.3	Kết quả benchmark – v1 vs v8	38
5	Source Code Review – v8	38
5.1	pipealloc()	38
5.2	pipeclose()	39
5.3	pipewrite()	40
5.4	piperead()	41
6	Phân tích tổng thể	42
6.1	Định luật Amdahl	42
6.1.1	Áp dụng Định luật Amdahl	43
6.2	Bài học kinh nghiệm (Methodology)	44
6.3	Hạn chế và hướng mở rộng	44
7	Phân công công việc	45

Phân công công việc & Đóng góp

STT	Thành viên	Nội dung công việc	Tiến độ
1	Nguyễn Sỹ Tuấn	v2 – Bulk Copy	11%
2		v4 – Multi-page Allocation	15%
3		v8 – All Combined (tích hợp, kiểm thử toàn bộ)	15%
4		Phân tích baseline v1, viết <code>pipebench.c</code> và báo cáo	4%
5	Lê Huy Sơn	v3 – Ring of Buffers	15%
6		v6 – Priority Boost	15%
7		v8 – All Combined (tích hợp v3, v6)	13%
8	Vũ Tiến Long	v5 – Lazy Wakeup	15%
9		v7 – Cache-line Alignment	15%
10		v8 – All Combined (tích hợp v5, v7)	3%
Tổng			100%

1 Giới thiệu

1.1 Đặt vấn đề

Pipe là cơ chế IPC cơ bản nhất trong hệ thống UNIX, được sử dụng rộng rãi để kết nối các tiến trình thông qua mô hình Producer-Consumer. Trong xv6 – hệ điều hành giáo dục viết bằng C cho kiến trúc RISC-V – pipe được triển khai đơn giản với bộ đệm 512 byte và cơ chế sao chép từng byte (byte-by-byte). Mặc dù đơn giản và dễ hiểu, cách triển khai này bỏ lại nhiều tiềm năng hiệu năng chưa được khai thác.

Dự án này đặt ra mục tiêu:

- **Bài toán:** Tối ưu (Optimization) – tối ưu IPC throughput (băng thông của Inter-Process Communication).
- **Phương thức:** Sửa đổi file `kernel/pipe.c` trong xv6-riscv.
- **Đầu vào/ra:** Do pipe là Message-Passing và truyền theo byte stream, đơn vị đo là Byte/s (hoặc MB/s, KB/s).
- **Hệ điều hành:** xv6-riscv, chạy trên QEMU với CPUS=1.

1.2 Câu hỏi nghiên cứu

Nhóm đặt ra 3 câu hỏi nghiên cứu ban đầu:

1. Input/Output của bài toán là gì?
2. Trong cơ chế IPC, pipe là gì và Pipe implementation làm những gì?
3. Hệ điều hành nào sẽ sử dụng (Windows/Linux/xv6...)?

Trả lời: (1) Tốc độ byte/giây. (2) Pipe là bộ đệm FIFO unidirectional, triển khai gồm: circular buffer, pipealloc, pipeclose, piperead, pipewrite. (3) xv6 vì đây là hệ điều hành mã nguồn mở giáo dục, cho phép sửa đổi kernel trực tiếp.

2 Cơ sở lý thuyết

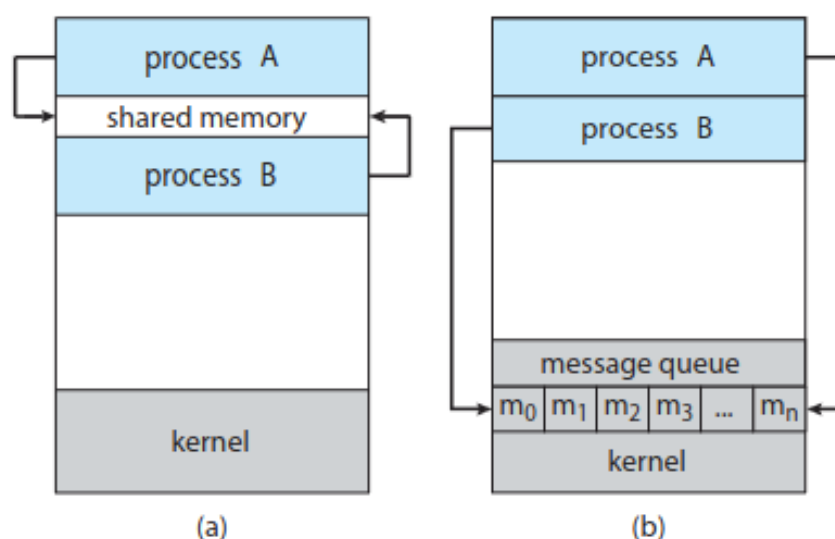
2.1 Inter-Process Communication (IPC)

Các tiến trình đang thực thi gồm 2 loại:

- **Independent:** không chia sẻ dữ liệu với các tiến trình khác.
- **Cooperating:** có sự chia sẻ dữ liệu, cần cơ chế IPC để trao đổi.

Các mô hình IPC chính:

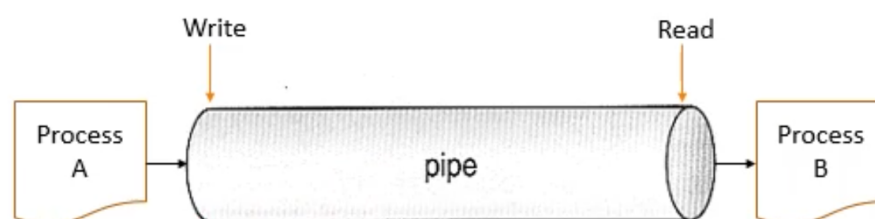
1. **Shared-Memory:** Có một vùng bộ nhớ được chia sẻ bởi các tiến trình cooperate, nằm trên không gian địa chỉ của tiến trình tạo ra vùng bộ nhớ đó. Các tiến trình gắn memory segment này vào không gian địa chỉ của chúng để giao tiếp. Đơn vị đo throughput: MB/s.
2. **Message-Passing:** Cho phép các tiến trình giao tiếp và đồng bộ các hoạt động qua message queue trong kernel. Giao tiếp cần thiết lập link theo 3 cách: (a) Direct or indirect thông qua các operations send(message), receive(message); (b) Synchronous or asynchronous – thông qua các hoạt động blocking và nonblocking; (c) Automatic or explicit buffering – sử dụng các bộ đệm tạm thời để lưu trữ messages. **Pipe thuộc cơ chế Message-Passing.** Đơn vị đo: Byte/s.
3. **Signal:** Cơ chế thông báo sự kiện không đồng bộ giữa các tiến trình.



Hình 1: Hai cơ chế giao tiếp: a) Shared-Memory b) Message-Passing

2.2 Pipe trong UNIX/xv6

Pipe là một dạng IPC **unidirectional**, **half-duplex**, hiện thực dưới dạng **bounded buffer** trong kernel – minh họa kinh điển của **producer-consumer problem** (Dijkstra, 1965).



Hình 2: Unix pipe

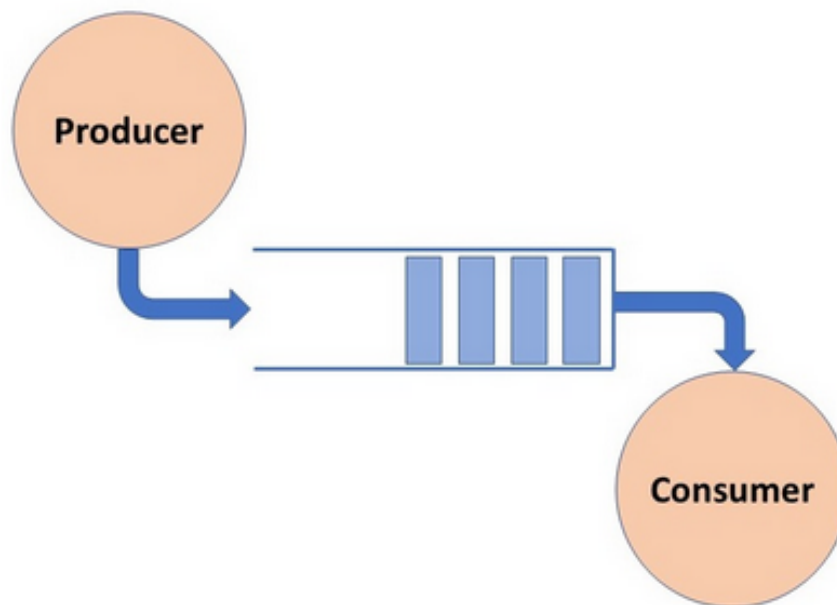
-
- + Unidirectional: Dữ liệu chỉ được truyền theo một hướng.
 - + Half-duplex: Tại một thời điểm, chỉ có một tiến trình được đọc hoặc ghi.
 - + Bounded buffer: Pipe có kích thước cố định, khi đầy tiến trình ghi sẽ bị chặn, khi rỗng tiến trình đọc sẽ bị chặn.

producer - consumer problem, hay còn gọi là bài toán Bounded-Buffer (Bộ đệm có kích thước giới hạn), là một trong những bài toán kinh điển nhất trong khoa học máy tính về đồng bộ hóa đa tiến trình (multi-process synchronization). Nó được Edsger W. Dijkstra đưa ra vào năm 1965. Hãy tưởng tượng một hệ thống có ba thành phần chính:

Producer (Người sản xuất): Một hoặc nhiều tiến trình (process/thread) liên tục tạo ra dữ liệu mới.

Consumer (Người tiêu dùng): Một hoặc nhiều tiến trình liên tục lấy dữ liệu đó ra để xử lý.

Buffer (Bộ đệm): Một vùng nhớ dùng chung (shared memory) có kích thước cố định (N phần tử) để chứa dữ liệu do Producer tạo ra trước khi Consumer kịp lấy đi.



Hình 3: Producer - Consumer Problem

Bởi vì cả Producer và Consumer đều chạy song song (concurrently) và cùng thao tác trên một bộ đệm chung, các vấn đề nghiêm trọng sẽ xảy ra nếu không có cơ chế kiểm soát:

Vượt quá giới hạn: Producer không được phép thêm dữ liệu khi bộ đệm đã đầy (phải ngủ/chờ đến khi có chỗ trống).

Lấy dữ liệu rác: Consumer không được phép lấy dữ liệu khi bộ đệm đang trống (phải ngủ/chờ đến khi có dữ liệu mới).

Race Condition: Nếu Producer và Consumer (hoặc hai Producer) cùng ghi/đọc vào một vị trí trong bộ đệm tại cùng một chu kỳ máy, dữ liệu sẽ bị ghi đè, sai lệch hoặc hỏng hóc.

Trong xv6, vấn đề này được giải quyết bằng cách sử dụng một trường khóa là *SPINLOCK*. Trong UNIX, pipe giống như một file: ta có thể ghi dữ liệu vào nó và đọc dữ liệu từ nó. Tuy nhiên, file được lưu trữ trên đĩa, còn dữ liệu trong pipe hoàn toàn nằm trên bộ nhớ.

Cấu trúc pipe baseline (v1):

```
1 #define PIPESIZE 512
2
3 struct pipe {
4     struct spinlock lock; // bao ve tat ca cac truong
5     char data[PIPESIZE]; // bo dem vong 512 bytes
6     uint nread;           // vi tri doc trong bo dem
7     uint nwrite;          // vi tri ghi trong bo dem
8     int readopen;         // tin hieu kích hoạt đọc
9     int writeopen;        // tin hieu kích hoạt ghi
10 };
```

Listing 1: struct pipe – v1 baseline, PIPESIZE = 512 byte

Giải thích các trường:

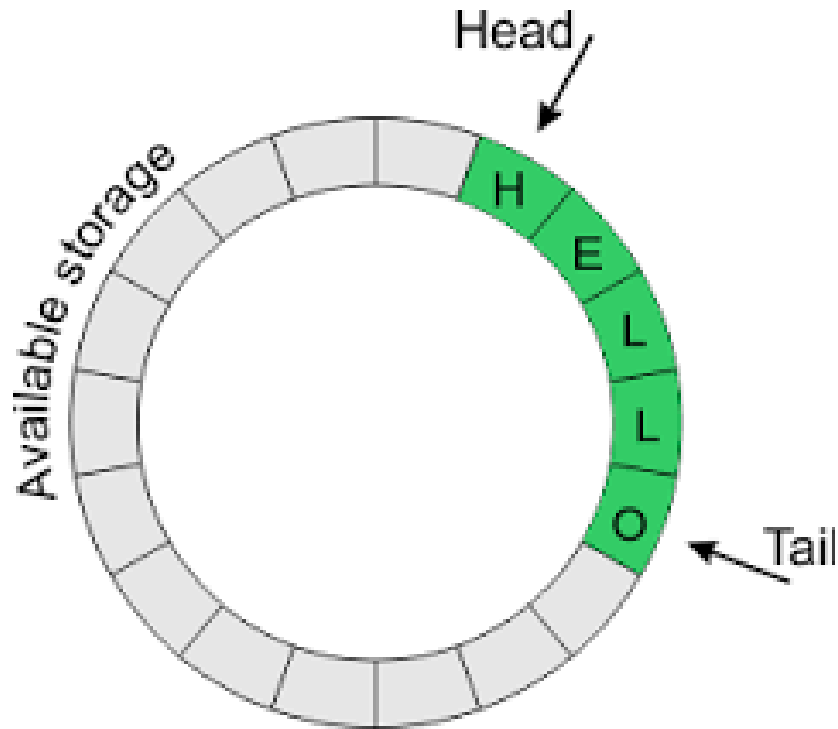
- **SPINLOCK:** bảo vệ tất cả các trường còn lại, ngăn chặn race condition.
- **nread, nwrite (uint):** chỉ số tích lũy – pipe **đầy** khi `nwrite == nread + PIPESIZE`, **rỗng** khi `nread == nwrite`. Truy cập vào bộ đệm vòng bằng modulo: `data[nwrite % PIPESIZE]`.
- **readopen, writeopen (int):** biến logic cho phép đọc/ghi.
- **data[] (512 byte):** bộ đệm vòng có kích thước `PIPESIZE = 512`.

Lãng phí bộ nhớ trong v1: Bất cứ khi nào cần tạo một pipe mới, kernel gọi `kalloc()` phân bổ 1 trang (`PAGESIZE = 4096` Byte). Các trường chiếm ~ 0.5 KB, trường trống còn lại là ~ 3.5 KB – không được sử dụng và lãng phí.

2.3 Circular Buffer (Bộ đệm vòng)

Pipe sử dụng bộ đệm vòng với cấu trúc như sau:

- Writer advances HEAD pointer (nwrite) sau khi ghi dữ liệu.
- Reader advances TAIL pointer (nread) sau khi đọc dữ liệu.
- Khi pointer đạt cuối mảng, nó “vòng lại” vị trí đầu (wrap-around).
- Buffer FULL khi `nwrite == nread + PIPESIZE`.
- Buffer EMPTY khi `nread == nwrite`.



Hình 4: Circular Buffer

2.4 Cơ chế Spinlock và Sleep/Wakeup trong xv6

Trước khi đến với vấn đề Producer - Consumer problem, thì ta đến với tiền thân của nó là Critical - Section problem. Cấu trúc của một tiến trình:

```

1  while(true){
2      entry section: // Yêu cầu vào critical section
3      critical section
4      exit section: // Thông báo kết thúc critical section
5      remainder section
6  }
```

Trong hệ thống pipe của xv6, gồm 2 tiến trình P_0 và P_1 , giả sử P_0 là tiến trình ghi và P_1 là tiến trình đọc. Mỗi tiến trình có một phân đoạn mã, gọi là *critical section*, nơi nó có thể truy cập và thay đổi các dữ liệu dùng chung với các tiến trình khác. Để giải quyết Critical - Section Problem, ta phải thiết kế với nguyên tắc là khi một tiến trình thực thi đoạn mã *critical section* thì không có tiến trình khác có thể thực thi xen vào. Ba yêu cầu cần phải thỏa mãn:

- **Mutual exclusion:** Nếu tiến trình P_0 đang thực thi critical section, thì tiến trình P_1 không được phép thực thi critical section.
- **Progress:** Nếu không có tiến trình nào trong critical section và một số tiến trình muốn vào, thì chỉ có những tiến trình đang không thực thi remainder section mới được quyết định tiến trình nào sẽ vào critical section tiếp theo, và quyết định này không được trì hoãn vô thời hạn.
- **Bounded waiting:** Tiến trình chờ trong critical section không được phép vượt quá một số hữu hạn các tiến trình khác đã vào critical section trước đó.

Trong chương trình đã học về Synchronization Tools trong môn hệ điều hành, để giải quyết Critical - Section Problem, ta có thể sử dụng: Peterson's Solution, các Hardware Instructions như `test_and_set`, `compare_and_swap` và các cơ chế như `mutexlock`, `semaphore`, `monitor`, ...

Trong hệ điều hành xv6, Synchronization Tools mà xv6 sử dụng là spin-lock (một phiên bản của mutex locks), có cấu trúc được định nghĩa trong file `spinlock.h` như sau:

```

1  #ifndef _SPINLOCK_H_
2  #define _SPINLOCK_H_
3  // Mutual exclusion lock.
4  struct spinlock {
5      uint locked;           // Is the lock held?
6
7      // For debugging:
8      char *name;           // Name of lock.
9      struct cpu *cpu;      // The cpu holding the lock.
10 };
11
12 #endif

```

Tuy nhiên, do pipe phải trao đổi dữ liệu giữa hai tiến trình đọc và ghi, nên phải sử dụng thêm cơ chế sleep/wakeup để đồng bộ hóa hai tiến trình này. Đây là Producer - Consumer Problem, cơ chế sleep được định nghĩa trong file `proc.h` như sau:

```

1 void sleep(void *chan, struct spinlock *lk)
2 {
3     struct proc *p = myproc();
4
5     // Must acquire p->lock in order to
6     // change p->state and then call sched.
7     // Once we hold p->lock, we can be
8     // guaranteed that we won't miss any wakeup
9     // (wakeup locks p->lock),
10    // so it's okay to release lk.
11

```

```

12  acquire(&p->lock);  //DOC: sleeplock1
13  release(lk);
14
15  // Go to sleep.
16  p->chan = chan;
17  p->state = SLEEPING;
18
19  sched();
20
21  // Tidy up.
22  p->chan = 0;
23
24  // Reacquire original lock.
25  release(&p->lock);
26  acquire(lk);
27 }

```

Và cơ chế wakeup được định nghĩa trong file *proc.h* như sau:

```

1  // Wake up all processes sleeping on channel chan.
2  // Caller should hold the condition lock.
3  void wakeup(void *chan)
4  {
5      struct proc *p;
6
7      for(p = proc; p < &proc[NPROC]; p++) {
8          if(p != myproc()){
9              acquire(&p->lock);
10             if(p->state == SLEEPING && p->chan == chan) {
11                 p->state = RUNNABLE;
12             }
13             release(&p->lock);
14         }
15     }
16 }

```

2.5 Cách pipe hoạt động với tiến trình

Giả sử có một tiến trình P. Một tiến trình có thể mở một số tệp tại bất kì thời điểm nào. User code tham chiếu đến các tệp riêng lẻ bằng cách sử dụng một **bộ mô tả tệp** (file descriptor) – là một số nguyên 0, 1, 2,...mà kernel dùng làm chỉ mục (index) vào mảng `ofile[]` bên trong `struct proc`.

Trong cấu trúc của mỗi tiến trình, mảng `ofile[]` chứa các con trỏ đến các đối tượng tệp (`struct file`). Mỗi đối tượng tệp có trường `type` – ở đây mang giá trị `FD_PIPE` để chỉ ra đây là pipe – và một con trỏ tới đối tượng thực sự (`struct pipe`, `inode`, ...).

Trong trường hợp pipe, có 2 trường điều khiển truy cập: `readopen` và `writeopen`. Mỗi trường chỉ bằng 1 khi còn ít nhất một file descriptor trở tới đầu tương ứng của pipe.

Quá trình tạo và sử dụng pipe (minh họa với P, R, W):

1. Tiến trình P gọi `pipe(fd[])` – kernel cấp phát `struct pipe`, tạo 2 `struct file` (READABLE và WRITEABLE), tìm 2 ô trống trong `ofile[]` của P và trả về 2 file descriptor, ví dụ **fd=2** (read end) và **fd=4** (write end).
2. P gọi `fork()` để tạo 2 tiến trình con **R** (đọc) và **W** (ghi). Do `fork()` sao chép toàn bộ `ofile[]`, cả R và W đều kế thừa hai file descriptor trên và cùng trở vào một `struct pipe` duy nhất trong kernel.
3. Mỗi tiến trình đóng đầu mà nó không cần:
 - R gọi `close(fd=4) ⇒ ofile[4] = NULL` (đóng write end).
 - W gọi `close(fd=2) ⇒ ofile[2] = NULL` (đóng read end).
4. W gửi dữ liệu đến R bằng cách gọi `write(fd=4, buf, n)`: kernel gọi `pipewrite()` để sao chép dữ liệu từ user space vào bộ đệm vòng của pipe trong kernel. Nếu bộ đệm đầy, W `sleep()` chờ R lấy bớt dữ liệu ra.
5. R đọc dữ liệu bằng cách gọi `read(fd=2, buf, n)`: kernel gọi `piperead()` để sao chép dữ liệu từ bộ đệm vòng ra user space của R. Nếu bộ đệm rỗng, R `sleep()` chờ W ghi thêm.

2.6 Thông lượng IPC (IPC Throughput)

Thông lượng của pipe được tính theo công thức:

$$\text{Throughput} = \frac{\text{DataTransfer (byte)}}{\text{TimeTransfer (giây)}} = \frac{\text{total_bytes} \times \text{TICKS_PER_SEC}}{\text{elapsed_ticks}} \quad (1)$$

Với:

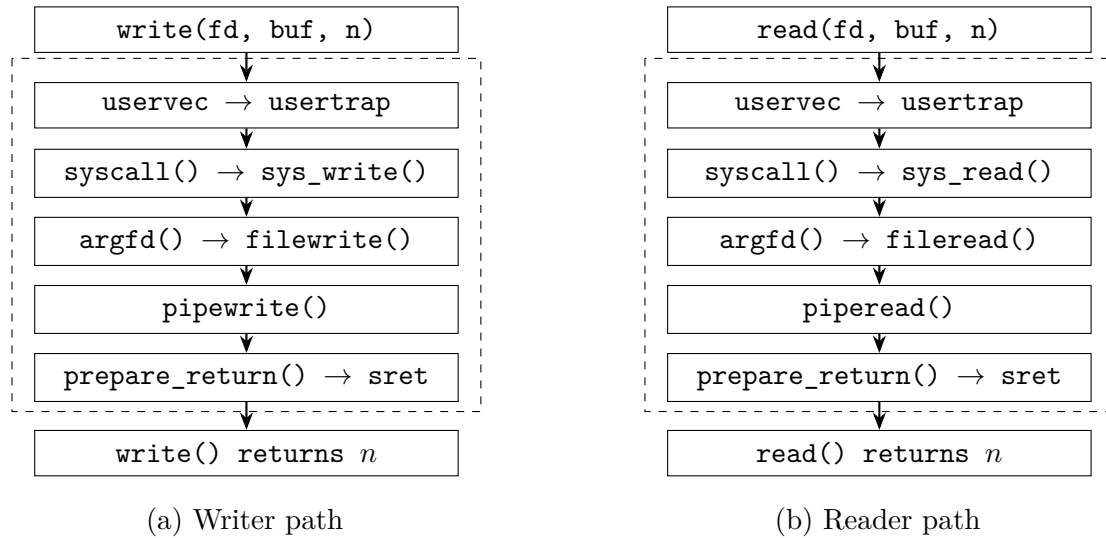
- **total_bytes**: là tổng số byte dữ liệu cần truyền qua pipe.
- **TICKS_PER_SEC**: là số tick trong 1 giây, xv6 sử dụng timer interrupt để tăng giá trị của tick lên 1.
- **elapsed_ticks**: là thời gian truyền hết dữ liệu qua pipe theo ticks.

Đơn vị đo chính: **Byte/s** (hoặc KB/s, MB/s). Lưu ý: 1 MB/s = 10^6 B/s còn 1 MiB/s = 2^{20} B/s $\approx 1.049 \times 10^6$ B/s – hai đơn vị này khác nhau về cơ số.

3 Phân tích Baseline (v1)

3.1 Đường đi của một lần write/read qua pipe

User process **không gọi trực tiếp** `pipewrite()/piperead()` mà gọi qua hàm `write()/read()` trong userspace và được định nghĩa trong file `sysfile.c`. Mỗi lần ghi/đọc trong đi qua một chuỗi các hàm trong kernel.



Hình 5: Đường đi `write()/read()` qua pipe trong xv6. Khung nét đứt = Kernel (Supervisor Mode); ô trắng = User space.

Phân tích chi tiết từng bước:

Bảng 1: Map từng hàm trong đường đi write/read pipe → file nguồn trong xv6

Hàm (Write)	Hàm (Read)	File	Dòng	Mô tả
ecall	ecall	ISA (RISC-V)	–	CPU trap sang supervisor mode; pc → stvec; lưu pc vào sepc
uservec	uservec	trampoline.S	:18	Lưu 32 regs vào trapframe; đổi satp sang kernel page table; nhảy tới usertrap()
usertrap()	usertrap()	trap.c	:38	Đọc scause=8 (syscall), gọi syscall(); xử lý interrupt/exception
syscall()	syscall()	syscall.c	:132	Đọc reg a7 (SYS_write=16 / SYS_read=63), dispatch bằng syscalls[]
sys_write()	sys_read()	sysfile.c	:83 / :69	argfd(), argaddr(), argint() đọc 3 args từ a0,a1,a2
argfd()	argfd()	sysfile.c	:22	Validate fd ∈ [0,NOFILE); trả về struct file* từ myproc()->ofile[fd]
filewrite()	fileread()	file.c	:135 / :107	Kiểm tra f->type == FD_PIPE; dispatch → pipewrite(f->pipe,...)
pipewrite()	piperead()	pipe.c	:243 / :465	acquire(&lock); vòng lặp: copyin/copyout từng byte, kiểm tra đầy/rỗng, sleep/wakeup
copyin()	copyout()	vm.c	:381 / :343	walkaddr() dịch VA→PA (Sv39 3-level), sau đó memmove copy dữ liệu
walkaddr()	walkaddr()	vm.c	:121	Duyệt 3 cấp PTE (9+9+9+12 bit), kiểm tra bit V/U/R/W; trả về PA
wakeup(&nread)	wakeup(&nwrite)	proc.c	:742	Quét proc[NPROC] (64 entry), đổi SLEEPING→RUNNABLE nếu chan khớp
prepare_return()	prepare_return()	trap.c	:100	Đặt sstatus.SPP=0 (user mode), sepc=epc; set stvec→uservec
userret→sret	userret→sret	trampoline.S	:101	Khôi phục 32 regs; đổi satp→user PT; sret: supervisor→user

⇒ Throughput không chỉ phụ thuộc vào kernel/pipe.c, mà còn phụ thuộc vào rất nhiều yếu tố khác: trap, syscall dispatch, file layer, copy memory, page table walk, TLB, cache, spinlock, sleep/wakeup, scheduler.

3.2 Source code đầy đủ – pipealloc, pipewrite và piperead v1

Source code sau đây là bản gốc xv6 tại PIPE_VERSION == 1, trích trực tiếp từ kernel/pipe.c. Đây là điểm xuất phát của toàn bộ chuỗi tối ưu.

3.2.1 struct pipe và pipealloc() v1

Cấu trúc dữ liệu struct pipe v1 (kernel/pipe.c:43):

```

1 #define PIPESIZE 512
2
3 struct pipe {
4     struct spinlock lock;    /* bao ve toan bo struct */
5     char data[PIPESIZE];    /* circular buffer 512 byte*/
6     uint nread;              /* so byte da doc */
7     uint nwrite;             /* so byte da ghi */

```

```

8  int  readopen;          /* 1 neu read-end con mo */
9  int  writeopen;         /* 1 neu write-end con mo */
10 };

```

Listing 2: struct pipe v1 – buffer nhúng 512 byte (kernel/pipe.c:43)

pipealloc() được gọi từ **sys_pipe()** khi user gọi syscall **pipe(int[2])**:

```

1  int
2  pipealloc(struct file **f0, struct file **f1)
3  {
4      // Khoi tao mot cau truc pipe *pi
5      struct pipe *pi;
6
7      // Khoi tao gia tri ban dau cho pipe la trong
8      pi = 0;
9      // Con tro tro toi cac file duoc gan ban dau la 0 de khong tro den file
      nao
10     *f0 = *f1 = 0;
11     // Ham filealloc() (nam trong kernel/file.c) tra ve con tro file neu
      thanh cong,
12     // tra ve 0 neu that bai. Neu mot trong hai file that bai, di den nhan
      bad
13     if((*f0 = filealloc()) == 0 || (*f1 = filealloc()) == 0)
14         goto bad;
15     // Ham kalloc() (nam trong kernel/kalloc.c): cap phat mot page 4096
      bytes,
16     // tra ve con tro neu thanh cong, tra ve 0 neu that bai.
17     // Neu cap phat khong thanh cong, di den nhan bad
18     if((pi = (struct pipe*)kalloc()) == 0)
19         goto bad;
20     // Cap phat thanh cong: gan cac truong readopen, writeopen = 1 (ca hai
      dau deu mo),
21     // vi tri ghi == vi tri doc == 0 (buffer rong)
22     pi->readopen = 1;
23     pi->writeopen = 1;
24     pi->nwrite = 0;
25     pi->nread = 0;
26     // Khoi tao khoa lien ket voi pipe bang ham initlock()
27     initlock(&pi->lock, "pipe");
28     // Gan gia tri cho cac file doc va ghi.
29     // Mot struct file (trong file.c) gom: type (FD_NONE/FD_PIPE/FD_INODE),
30     // ref, readable, writable, struct pipe*, struct inode*, off
31     (*f0)->type = FD_PIPE; // f0: read-end
32     (*f0)->readable = 1;
33     (*f0)->writable = 0;
34     (*f0)->pipe = pi;
35     (*f1)->type = FD_PIPE; // f1: write-end

```

```

36 (*f1)->readable = 0;
37 (*f1)->writable = 1;
38 (*f1)->pipe = pi;
39 return 0;
40
41 bad:
42 // Nhan bad: giai phong trang bang kfree(), dong cac file bang
    fclose()
43 if(pi) kfree((char*)pi);
44 if(*f0) fclose(*f0);
45 if(*f1) fclose(*f1);
46 return -1;
47 }

```

Listing 3: pipealloc() – v1, khai tao pipe (kernel/pipe.c:151)

Nhận xét pipealloc() v1 – 2 điểm đáng chú ý:

1. kalloc() lãng phí: kalloc() cấp phát đúng 1 trang = 4096 byte. Nhưng struct pipe v1 chỉ chiếm ≈ 560 byte (spinlock $\approx 40B$ + data[512] + $4 \times \text{uint}$ + $2 \times \text{int}$). Phần còn lại $\approx 3,536$ byte trong trang bị lãng phí hoàn toàn.

3.2.2 pipewrite() và piperead() v1

```

1 int
2 pipewrite(struct pipe *pi, uint64 addr, int n)
3 {
4     int i = 0;
5     struct proc *pr = myproc();
6
7     // Lay khoa truooc khi vao vung critical section
8     acquire(&pi->lock);
9     // Ghi tung byte du lieu vao bo dem
10    while(i < n){
11        // Neu read-end da dong hoac tien trinh bi ket lieu, ghi that bai
12        if(pi->readopen == 0 || killed(pr)){
13            release(&pi->lock);
14            return -1;
15        }
16        // Bo dem day (nwrite - nread == PIPESIZE): danh thuc reader dang cho
17        // sau do tien trinh ghi chuyen sang che do ngu cho den khi co cho
18        // trong
19        if(pi->nwrite == pi->nread + PIPESIZE){
20            wakeup(&pi->nread);
21            sleep(&pi->nwrite, &pi->lock);
22        } else {

```

```

22     char ch;
23     // Doc 1 byte tu user address space vao bien tam trong kernel
24     if(copyin(pr->pagetable, &ch, addr + i, 1) == -1)
25         break;
26     // Ghi byte vao vi tri hien tai trong bo dem vong, tang nwrite
27     pi->data[pi->nwrite++ % PIPESIZE] = ch;
28     i++;
29 }
30 }
31 // Ghi xong: danh thuc tien trinh doc de doc tiep, tra lai khoa
32 wakeup(&pi->nread);
33 release(&pi->lock);
34 return i;
35 }

```

Listing 4: pipewrite() – v1 baseline, byte-by-byte (kernel/pipe.c:243)

```

1  int
2  piperead(struct pipe *pi, uint64 addr, int n)
3  {
4      int i;
5      struct proc *pr = myproc();
6      char ch;
7
8      // Lay khoa truoc khi vao vung critical section
9      acquire(&pi->lock);
10     // Kiem tra xem pipe co rong va write-end con mo hay khong.
11     // Neu rong va writer van con, chuyen sang che do ngu cho den khi co du
        lieu
12     while(pi->nread == pi->nwrite && pi->writeopen){
13         // Kiem tra tien trinh co bi ket lieu hay chua, neu bi ket lieu tra
        ve -1
14         if(killed(pr)){
15             release(&pi->lock);
16             return -1;
17         }
18         // Bo dem rong: chuyen sang che do ngu, giai phong khoa cho writer
        ghi
19         sleep(&pi->nread, &pi->lock);
20     }
21     // Doc tung byte du lieu tu bo dem
22     for(i = 0; i < n; i++){
23         // Da doc het du lieu trong bo dem, dung
24         if(pi->nread == pi->nwrite)
25             break;
26         // Doc byte tai vi tri hien tai trong bo dem vong, tang nread
27         ch = pi->data[pi->nread++ % PIPESIZE];
28         // Ghi 1 byte tu bien tam trong kernel ra user address space

```

```

29     if(copyout(pr->pagetable, addr + i, &ch, 1) == -1)
30         break;
31 }
32 // Doc xong: danh thuc tien trinh ghi dang cho, tra lai khoa
33 wakeup(&pi->nwrite);
34 release(&pi->lock);
35 return i;
36 }

```

Listing 5: `piperead()` – v1 baseline, byte-by-byte (`kernel/pipe.c:465`)

3.2.3 `pipeclose()` v1

`pipeclose()` được gọi khi tiến trình đóng một đầu pipe (qua syscall `close()`). Hàm nhận tham số `writable` để phân biệt đóng write-end (`writable=1`) hay read-end (`writable=0`), đặt cờ tương ứng về 0, đánh thức bên đối diện nếu đang ngủ, và giải phóng trạng vật lý khi cả hai đầu đều đã đóng:

```

1 void
2 pipeclose(struct pipe *pi, int writable)
3 {
4     // writable=1: dong write-end; writable=0: dong read-end
5     // Lay khoa cho doi tuong pipe
6     acquire(&pi->lock);
7     // Dong viec ghi: dat writeopen = 0
8     if(writable){
9         pi->writeopen = 0;
10        // Danh thuc bat ki tien trinh doc nao dang cho, bao hieu write-end
        // da dong
11        wakeup(&pi->nread);
12        // Dong viec doc: dat readopen = 0
13    } else {
14        pi->readopen = 0;
15        // Danh thuc bat ky tien trinh ghi nao dang cho, bao hieu read-end da
        // dong
16        wakeup(&pi->nwrite);
17    }
18    // Neu ca hai dau da dong -> hoan thanh: tra lai khoa va giai phong
    // trang bo nho.
19    // Neu chua thi chi tra lai khoa, giu nguyen pipe cho dau kia tiep tục
    // dung
20    if(pi->readopen == 0 && pi->writeopen == 0){
21        release(&pi->lock);
22        kfree((char*)pi);
23    } else
24        release(&pi->lock);
25 }

```

Listing 6: `pipeclose()` – v1, dòng mã đầu pipe (`kernel/pipe.c:201`)

Nhận xét source code v1 – 3 vấn đề chính:

1. **byte-by-byte copy:** Mỗi byte = 1 lần `copyin/copyout` = 1 lần `walkaddr()`. Với 4096 byte, cần 4096 lần gọi `walkaddr()` do đó chi phí gọi syscall là rất cao.
2. **wakeup vô điều kiện:** Dòng cuối `pipewrite` gọi `wakeup(&pi->nread)` dù reader có thể *không ngủ* – quét 64 entry `proc[]` lãng phí thời gian quét mỗi lần ghi.
3. **PIPESIZE = 512:** Buffer chỉ 512 byte. Một lần ghi chunk-bytes = 4096 bị chia 8 đợt, mỗi đợt khi buffer đầy writer phải `sleep/wakeup` để làm rỗng bộ đệm, dẫn đến giảm băng thông.

Từ các phân tích ở các mục trên, các yếu tố tác động đến băng thông là:

1. **Chi phí gọi syscall.**
2. **Kích thước pipe buffer:** 512 bytes quá bé, cần tăng kích thước bộ đệm.
3. **copyin/copyout:** cần có cách để copy một lúc nhiều bytes chứ không phải 1 byte.
4. **Spinlock & sleep/wakeup:** Cần có cơ chế synchronization tốt hơn.

Ngoài ra, còn một số yếu tố khác cũng ảnh hưởng đến băng thông:

1. **Scheduler:** xv6 sử dụng cơ chế lập lịch Round-Robin, có cơ chế lập lịch nào tốt hơn không?
2. **CPU cache:** Hầu hết các dòng CPU hiện đại (Intel, AMD, ARM) đều có cacheline là 64 bytes, nếu các dữ liệu của writer và reader nằm trong cùng một cacheline thì sẽ xảy ra false sharing.

Tổng hợp các kiến thức trên, một lần truyền n byte qua pipe trong v1 tốn:

$$\begin{aligned} T_{v1}(n) = & T_{\text{syscall_write}} \\ & + n * T_{\text{copyin}}(1) \quad (\text{mỗi byte 1 lần copyin}) \\ & + n * T_{\text{lock_acquire_release}} \\ & + n * T_{\text{check}} \\ & + \text{ceil}(n/\text{PIPESIZE}) * T_{\text{sleep_wakeup}} \\ & + (\text{đội xung cho reader}) \end{aligned}$$

Nhận xét về `chunk_bytes`: `chunk_bytes` càng lớn thì số lần gọi syscall càng ít, giúp giảm overhead trap vào kernel. Tuy nhiên, chunk lớn hơn đồng nghĩa mỗi lần `write()` phải chờ reader drain buffer nhiều lần hơn, làm tăng chi phí `sleep()/wakeup()` trên mỗi lần gọi – đây là sự đánh đổi giữa ít syscall và nhiều lần chờ đồng bộ. Do đó trong bài báo cáo này, chúng tôi sẽ đo bằng thông từng version với các `chunk_bytes` là 64, 128, 256, 512, 1024, 2048 và 4096 để xem bằng thông thay đổi như nào.

3.3 Các phiên bản tối ưu thông lượng

Sau khi đã phân tích các yếu tố ảnh hưởng đến bằng thông, chúng tôi đề xuất các phiên bản tối ưu thông lượng dựa trên các báo cáo tìm hiểu hàng tuần:

Bảng 2: Các phiên bản tối ưu thông lượng

#	Phiên bản tối ưu
1	v2 – Bulk Copy
2	v3 – Ring of Buffers
3	v4 – Multi-page
4	v5 – Lazy Wakeup
5	v6 – Priority Boost
6	v7 – Cache-line Align
7	v8 – All Combined

3.4 Đo bằng thông của phiên bản v1

Để đo được bằng thông của phiên bản v1, chúng tôi đã viết ra chương trình C tại thư mục `user/pipebench.c`, chương trình này sẽ đo bằng thông của pipe bằng cách tạo ra 2 tiến trình, tiến trình cha (parent) ghi và tiến trình con (child) đọc. Dữ liệu tổng truyền đi mặc định có độ lớn là 4MiB bytes. Với `chunk_bytes` là kích thước mỗi lần `write()/read()`. Để chạy chương trình này:

```
1  make clean
2  make PIPE_VERSION=1 qemu
3  # Sau khi QEMU chạy, trong xv6:
4  pipebench                # quiet chunk_bytes, total mặc định ~4
5  MB
6  pipebench chunk [total_bytes] # quiet chunk, co định total
7  pipebench total [chunk_bytes] # quiet total, co định chunk
8  # Ví dụ:
9  pipebench chunk 1048576      # quiet chunk voi total = 1 MB
   pipebench total 256         # quiet total voi chunk = 256 B
```

Sau khi chạy xong, chúng tôi sẽ thu được kết quả trong file *ipctp.txt* trong filesystem xv6. Để xem kết quả, chúng tôi có thể dùng lệnh `cat ipctp.txt` trong xv6.

Bảng 3: Throughput baseline v1 (B/s) theo chunk_bytes

chunk_bytes	v1 (B/s)
64	198,782
128	310,689
256	419,430
512	511,500
1024	511,500
2048	559,240
4096	559,240

4 Các phiên bản tối ưu (v2–v8)

4.1 v2 – Bulk Copy

Thay vì mỗi lần gọi `copyin()/copyout()` ta chỉ ghi/đọc được 1 byte, ta có thể ghi/đọc 1 khối `chunk_bytes` với mỗi lần `copyin()/copyout()` do đó chi phí $T_{v2}(n)$ sẽ giảm đi, thay vì là $n \cdot T_{\text{copyin}}(1)$ sẽ là $\lceil n/\text{chunk_bytes} \rceil \cdot T_{\text{copyin}}(\text{chunk_bytes})$. Giải pháp này được gọi là Bulk Copy.

```

1 // v2 - pipewrite:
2 int
3 pipewrite(struct pipe *pi, uint64 addr, int n)
4 {
5     int i = 0;
6     struct proc *pr = myproc();
7
8     acquire(&pi->lock);
9     while(i < n){
10         if(pi->readopen == 0 || killed(pr)){
11             release(&pi->lock);
12             return -1;
13         }
14         if(pi->nwrite == pi->nread + PIPESIZE){
15             wakeup(&pi->nread);
16             sleep(&pi->nwrite, &pi->lock);
17         } else {
18             uint free_slots = PIPESIZE - (pi->nwrite - pi->nread);
19             uint head_slot = pi->nwrite % PIPESIZE;
20             uint till_wrap = PIPESIZE - head_slot;

```

```

21     int to_copy = (int)umin((uint)(n - i), umin(free_slots, till_wrap))
    ;
22     if(copyin(pr->pagetable, &pi->data[head_slot], addr + i, to_copy)
    == -1)
23         break;
24     pi->nwrite += to_copy;
25     i         += to_copy;
26 }
27 }
28 wakeup(&pi->nread);
29 release(&pi->lock);
30 return i;
31 }

```

Listing 7: v2 – pipewrite thay byte-by-byte bằng bulk copyin

```

1 // v2 - piperead:
2 int
3 piperead(struct pipe *pi, uint64 addr, int n)
4 {
5     int i = 0;
6     struct proc *pr = myproc();
7
8     acquire(&pi->lock);
9     while(pi->nread == pi->nwrite && pi->writeopen){
10         if(killed(pr)){
11             release(&pi->lock);
12             return -1;
13         }
14         sleep(&pi->nread, &pi->lock);
15     }
16     while(i < n){
17         if(pi->nread == pi->nwrite)
18             break;
19         uint avail      = pi->nwrite - pi->nread;
20         uint tail_slot = pi->nread % PIPESIZE;
21         uint till_wrap = PIPESIZE - tail_slot;
22         int to_copy = (int)umin((uint)(n - i), umin(avail, till_wrap));
23         if(copyout(pr->pagetable, addr + i, &pi->data[tail_slot], to_copy) ==
        -1){
24             if(i == 0) i = -1;
25             break;
26         }
27         pi->nread += to_copy;
28         i         += to_copy;
29     }
30     wakeup(&pi->nwrite);
31     release(&pi->lock);

```

```
32     return i;
33 }
```

Listing 8: v2 – piperead thay byte-by-byte bằng bulk copyout

Hàm `umin(a, b)` trả về giá trị nhỏ hơn trong hai số nguyên không dấu. Tại vòng lặp `while()` sẽ ghi 1 khối `n = chunk_bytes` với các bước sau:

1. Kiểm tra xem buffer đã đầy chưa, điều kiện `pi->nwrite == pi->nread + PIPESIZE` sẽ được kiểm tra ở mỗi lần ghi. Nếu buffer đã đầy thì sẽ đánh thức reader và sleep writer.
2. Nếu chưa đầy thì tính toán số bytes còn trống trong buffer, vị trí con trỏ head bằng phép modulo vì `nwrite` có thể lớn hơn giá trị buffer hiện tại.
3. Tính toán `till_wrap` bytes có thể ghi trong 1 lần tính từ head đến cuối bộ đệm.
4. So sánh số bytes có thể ghi vào, lấy giá trị nhỏ nhất bằng hàm `umin()`. Có 3 trường hợp có thể xảy ra:
 - (a) Số bytes còn lại cần ghi là nhỏ nhất: `to_copy = n - i`.
 - (b) Số bytes còn trống trong buffer lớn hơn số byte có thể ghi trong 1 lần, xảy ra khi dữ liệu đã được đọc trong `piperead`: `to_copy = till_wrap`.
 - (c) Số bytes còn trống trong buffer nhỏ hơn số bytes có thể ghi trong 1 lần, xảy ra khi vẫn còn dữ liệu cũ chưa được đọc: `to_copy = free_slots`.
5. Gọi `copyin()` để ghi khối dữ liệu vào buffer.
6. Cập nhật con trỏ `nwrite` và `i`.

Tại vòng lặp `while()` sẽ đọc 1 khối `n = chunk_bytes` với các bước sau:

1. Kiểm tra xem buffer đã rỗng chưa, điều kiện `pi->nread == pi->nwrite` sẽ được kiểm tra ở mỗi lần đọc. Nếu buffer đã rỗng thì sẽ sleep reader.
2. Nếu chưa rỗng thì tính toán số bytes có thể đọc trong 1 lần tính từ tail đến cuối bộ đệm.
3. So sánh số bytes có thể đọc vào, lấy giá trị nhỏ nhất bằng hàm `umin()`. Có 3 trường hợp có thể xảy ra:
 - (a) Số bytes còn lại cần đọc là nhỏ nhất: `to_copy = n - i`.
 - (b) Số bytes còn trống trong buffer lớn hơn số byte có thể đọc trong 1 lần, xảy ra khi dữ liệu đã được ghi trong `pipewrite`: `to_copy = till_wrap`.

-
- (c) Số bytes còn trống trong buffer nhỏ hơn số bytes có thể đọc trong 1 lần, xảy ra khi vẫn còn dữ liệu cũ chưa được ghi: `to_copy = avail`.

4. Gọi `copyout()` để đọc khối dữ liệu từ buffer.

5. Cập nhật con trỏ `nread` và `i`.

Ưu điểm:

- Giảm số lần gọi `copyin()/copyout()` từ n xuống $\lceil n/\text{chunk_bytes} \rceil$.
- Không thay đổi cấu trúc dữ liệu.

Nhược điểm:

- Vẫn bị giới hạn bởi `PIPESIZE = 512` byte.
- Vẫn còn wrap-around: một lần ghi có thể bị cắt thành 2 lần `copyin` ở biên vòng tròn 512 byte.
- Critical section dài hơn (giữ spinlock trong khi `copyin()/copyout()` cả khối).

4.1.1 Kết quả đo đạc v2

Tương tự với `v1`, để đo băng thông của `v2`, ta cũng chạy như sau:

```
1  make clean
2  make PIPE_VERSION=2 qemu // Ung voi v2
3  \ \ Sau khi QEMU ạych, trong xv6, ạych ệnh:
4  pipebench
```

So sánh băng thông của `v1` và `v2`:

Bảng 4: So sánh throughput `v1` vs `v2` (B/s)

<code>chunk_bytes</code>	<code>v1</code> (B/s)	<code>v2</code> (B/s)	Tăng
64	198,782	268,865	1.35×
128	310,689	499,321	1.61×
256	419,430	855,980	2.04×
512	511,500	1,398,101	2.73×
1024	511,500	1,613,193	3.15×
2048	559,240	1,747,626	3.13×
4096	559,240	1,823,610	3.26×

Ta thấy sự tăng trưởng giữa `v1` và `v2` là khá lớn, đặc biệt là ở `chunk_bytes` lớn, lý do như sau:

- Với `chunk_bytes` nhỏ, số lần gọi syscall sẽ nhiều (số lần gọi syscall = `total_bytes / chunk_bytes`), do đó băng thông bị giới hạn bởi thời gian xử lý syscall.
- Với `chunk_bytes` lớn, số lần gọi syscall đã giảm đi, do đó băng thông giờ theo chi phí số lần `copyin()/copyout()`.

4.2 v3 – Ring of Buffers

Tuy nhiên v2 vẫn bị giới hạn bởi 512 bytes, do đó, khi gọi một `chunk_bytes` lớn hơn so với bộ đệm thì bộ đệm sẽ thường xuyên bị đầy, và do đó cần gọi nhiều lần `sleep()/wakeup()`. Do đó, ta cần tăng dung lượng bộ đệm lên.

Một lý do nữa là khi ta cấp một trang cho pipe thì có tận 3.5K bytes không được sử dụng dẫn đến bị lãng phí dữ liệu không được sử dụng đó. Để tăng dung lượng bộ đệm ta thay đổi trường `data[]` trong trường pipe như sau, giả sử tăng lên 2048 bytes:

```

1 #define PIPESIZE 2048
2
3 struct pipe {
4     struct spinlock lock;    /* bao ve toan bo struct */
5     char data[PIPESIZE];    /* circular buffer 2048 bytes*/
6     uint nread;              /* so byte da doc */
7     uint nwrite;             /* so byte da ghi */
8     int readopen;            /* 1 neu read-end con mo */
9     int writeopen;           /* 1 neu write-end con mo */
10 };

```

So sánh v2 (PIPESIZE=512) và v2-naive khi tăng PIPESIZE lên 2048 byte:

Bảng 5: So sánh throughput v2 (PIPESIZE=512) vs v2-naive (PIPESIZE=2048) (B/s)

chunk_bytes	v2-512 (B/s)	v2-2048 (B/s)	Tăng
64	268,865	283,398	1.05×
128	499,321	551,882	1.11×
256	855,980	1,048,576	1.22×
512	1,398,101	1,997,287	1.43×
1024	1,613,193	3,495,253	2.17×
2048	1,747,626	5,242,880	3.00×
4096	1,823,610	5,991,862	3.29×

Tuy nhiên, một vấn đề mới lại xảy ra, khi `nwrite` gần cuối mảng `data[PIPESIZE]`, một lần ghi `to_copy` byte có thể vượt biên cuối của bộ đệm do đó, kernel buộc phải gọi `copyin` hai lần, như ví dụ ở dưới:

v2 ring (PIPESIZE = 512), nwrite gần cuối:

```
+-----+
| .....[head=500]      |
+-----+
```

ghi 100 byte chia làm 2:

copyin #1: 12 byte (500..512)

copyin #2: 88 byte (0..88)

4.2.1 Giải pháp – Chunked 2D Ring Buffer

```
1 #define N_BUFS    4
2 #define BUFSIZE   512
3 #define PIPE_TOTAL (N_BUFS * BUFSIZE)    // 2048 byte tong
4
5 struct pipe {
6     struct spinlock lock;
7     char data[N_BUFS][BUFSIZE]; // 2D array -- thay char data[PIPESIZE]
8     uint nread;
9     uint nwrite;
10    int readopen;
11    int writeopen;
12 };
13
14 // Indexing moi:
15 uint pos      = pi->nwrite % PIPE_TOTAL;
16 uint buf_idx  = pos / BUFSIZE;           // chi so sub-buffer (0-3)
17 uint buf_off  = pos % BUFSIZE;           // vi tri trong sub-buffer
18 uint till_buf_end = BUFSIZE - buf_off;   // byte con lai trong sub-
    buffer
19 int to_copy    = umin(n - i, umin(free_slots, till_buf_end));
20 copyin(pr->pagetable, &pi->data[buf_idx][buf_off], addr + i, to_copy);
```

Listing 9: v3 – struct pipe với 2D ring buffer ($N_BUFS \times BUFSIZE$)

Cơ chế: Mỗi lần copyin nằm gọn trong 1 sub-buffer – wrap-around chỉ xảy ra khi đổi sub-buffer (ranh giới nhỏ hơn). Liên hệ thực tế: Linux kernel dùng `pipe_inode_info` với 16 trang $\times$ 4096 byte mỗi trang; DPDK ring buffers dùng power-of-two sub-buffers.

4.2.2 Kết quả đo đạc v3

Bảng 6: So sánh throughput v2-naive (PIPE_SIZE=2048) vs v3 – Ring of Buffers (B/s)

chunk_bytes	v2-2048 (B/s)	v3 (B/s)	Tăng
64	283,398	281,496	0.99×
128	551,882	559,240	1.01×
256	1,048,576	1,048,576	1.00×
512	1,997,287	1,997,287	1.00×
1024	3,495,253	3,226,387	0.92×
2048	5,242,880	5,242,880	1.00×
4096	5,991,862	5,991,862	1.00×

Phân tích: v3 và v2-naive (cùng buffer 2048 byte) có throughput gần như bằng nhau – v3 thậm chí kém hơn nhẹ ở chunk=1024 (0.92×) do overhead của phép tính index 2D (buf_idx, buf_off). Điều này cho thấy việc tăng đơn giản PIPE_SIZE (v2-naive) đã đạt được hầu hết lợi ích của bộ đệm 2048 byte; giải pháp thực sự cần là **tăng kích thước bộ đệm đủ lớn** (v4: trang riêng 4096 byte).

4.3 v4 – Multi-page Allocation

Nhận thấy rằng ở trường pipe có thể chia ra làm 2 phần:

- Phần metadata: gồm spinlock, nread, nwrite, readopen, writeopen dùng để quản lý và theo dõi trạng thái của pipe.
- Phần data: gồm data[BUFSIZE] dùng để lưu trữ dữ liệu của pipe.

Do đó, thay vì cấp 1 trang cho pipe bao gồm cả metadata và data, ta cấp 2 trang riêng biệt cho metadata và data. Do đó ta sẽ vừa dễ quản lý bộ nhớ mà vừa dễ truy cập dữ liệu. Một lý do nữa là đối với các CPU hiện đại hiện nay (Intel, AMD, ARM) thì mỗi lần truyền dữ liệu từ RAM sang CPU cache mất 1 cache line là 64 bytes (bus 64 bits * Burst Length = 8), đối với v1 thì trường spinlock, data nằm trên cùng một cache line, mà khi truyền một dữ liệu lớn, cần gọi nhiều lần copyin()/copyout(), CPU phải load/store liên tục nhiều cache line vào các cache L1/L2 và xóa bỏ các cache line cũ dẫn đến line chứa các trường metadata bị đẩy ra khỏi cache và CPU phải lấy lại các trường này. Source code cấp 2 trang riêng biệt cho metadata và data như sau:

```
1 #if PIPE_VERSION == 4
2 #define PIPE_DATA_SIZE PGSIZE // dung tron 1 trang (4096 byte) cho data
3
4 struct pipe {
```

```

5     struct spinlock lock;
6     char *data;           // con tro toi trang data rieng (kalloc() lan 2)
7     uint nread;
8     uint nwrite;
9     int  readopen;
10    int  writeopen;
11};
12
13// pipealloc -- cap 2 trang:
14struct pipe *pi = (struct pipe*) kalloc(); // trang 1: struct
15pi->data = (char*) kalloc();              // trang 2: data buffer

```

Listing 10: Cấp 2 trang riêng biệt cho metadata và data

Layout bộ nhớ trước và sau v4:

Trước v4 (v3) -- metadata và data trên cùng 1 trang:

```

+-----+
| spinlock  (40 B) | cache line 0 |
| nread     ( 4 B) |               |
| nwrite    ( 4 B) |               |
| readopen  ( 4 B) +               |
| data[0][0..63]   | cache line 1 | <- ghi data có thể evict metadata!
+-----+

```

Sau v4 -- tách thành 2 trang riêng biệt:

Trang 1 (struct điều khiển)	Trang 2 (data buffer)
+-----+	+-----+
spinlock (40 B)	data[0..4095]
char *data (8 B)	+---> (4096 byte = 1 trang)
nread (4 B)	cache line A (0-63)
nwrite (4 B)	cache line B (64-127)
readopen (4 B)	...
writeopen (4 B)	cache line 63 (4032-)
(padding ~4032 B)	+-----+
+-----+	
(metadata biết lập)	(data biết lập)

4.3.1 Kết quả đo đạc v4

Bảng 7: So sánh throughput v3 vs v4 (B/s)

chunk_bytes	v3 (B/s)	v4 (B/s)	Tăng
512	1,997,287	2,097,152	1.05×
1024	3,226,387	3,813,003	1.18×
2048	5,242,880	5,991,862	1.14×
4096	5,991,862	10,485,760	1.75×

Ta thấy có một sự trade-off ở đây:

- Do tăng bộ đệm từ 2048 lên 4096 bytes, số lần gọi `copyin()/copyout()` giảm đi, do đó băng thông được tăng; data và metadata được tách riêng biệt.
- Tuy nhiên metadata chỉ có khoảng 64 bytes, trong khi 1 trang tận 4096 byte, do đó vừa làm tăng tài nguyên cấp mà vừa lãng phí ở metadata.

4.4 v5 – Lazy Wakeup

Trong các phiên bản từ v1 đến v4, cuối mỗi lần `pipewrite()` đều gọi `wakeup(&pi->nread)` vô điều kiện, và cuối mỗi `piperead()` đều gọi `wakeup(&pi->nwrite)` vô điều kiện. Nhìn vào source code `wakeup()` trong xv6, chứa trong file `kernel/proc.c`:

```
1 void wakeup(void *chan) {
2     struct proc *p;
3     for (p = proc; p < &proc[NPROC]; p++) { // quét 64 entries
4         if (p != myproc()) {
5             acquire(&p->lock);
6             if (p->state == SLEEPING && p->chan == chan)
7                 p->state = RUNNABLE;
8             release(&p->lock);
9         }
10    }
11 }
```

Listing 11: `wakeup()`

NPROC được định nghĩa trong file `kernel/param.h` là 64, nghĩa là số tiến trình chạy tối đa là 64. Mỗi lần `wakeup()`, quét tất cả 64 tiến trình, do đó mất thời gian $T_{sleep_wakeup} = (\text{acquire lock} + \text{load state} + \text{compare} + \text{release lock}) \times 64$, **kể cả khi không có process nào đang ngủ trên chan.**

Khi writer đang thực thi bên trong `pipewrite()` và giữ `pi->lock`, reader có thể ở một trong ba trạng thái:

- *State 1 – Reader đang SLEEPING trên `pi->nread`*: Xảy ra khi reader gọi `piperead()`, thấy buffer rỗng (`nread == nwrite`), rồi gọi `sleep(&pi->nread, &pi->lock)`. Bên trong `sleep()`, hàm giải phóng `pi->lock` trước khi đặt trạng thái SLEEPING và gọi `sched()` nhường CPU. Chính vì lock đã được giải phóng, writer mới có thể `acquire(&pi->lock)` và bắt đầu ghi. → `wakeup()` cần thiết để chuyển reader từ SLEEPING sang RUNNABLE.
- *State 2 – Reader đang RUNNABLE*: Xảy ra khi reader đã được `wakeup` từ trước (lần ghi trước đó đánh thức nó) nhưng scheduler chưa chọn nó chạy – reader đang xếp hàng chờ CPU. → `wakeup()` **KHÔNG cần thiết**: state đã là RUNNABLE, gọi thêm chỉ quét `proc[]` lãng phí.
- *State 3 – Reader đang RUNNING (chỉ trong multi-CPU)*: Đối với (single-CPU), tại bất kỳ thời điểm nào chỉ có một process nắm CPU. Khi writer đang thực thi và giữ `pi->lock`, reader không thể ở trạng thái RUNNING đồng thời – nó bắt buộc phải ở SLEEPING hoặc RUNNABLE. State 3 chỉ có ý nghĩa khi `CPUS ≥ 2`: Core 0 chạy writer, Core 1 chạy reader song song; nhưng khi đó reader đang giữ `pi->lock` nên writer phải spin chờ – không thể ghi. → `wakeup()` **KHÔNG cần thiết**.

→ Chỉ State 1 mới thực sự cần `wakeup()`. Để giải quyết vấn đề này có 2 giải pháp:

Giải pháp 1: Thêm các trường cờ vào struct pipe để track trạng thái reader/writer. Tuy nhiên, điều này làm tăng độ phức tạp code và tăng dung lượng cho metadata.

Giải pháp 2: reader chỉ gọi `sleep()` khi và chỉ khi buffer rỗng (`nread == nwrite`). Ngược lại, nếu buffer có dữ liệu khi writer bắt đầu ghi, reader hoặc đang RUNNABLE (đã được đánh thức từ trước), hoặc chưa cần đọc gì – trong cả hai trường hợp đều không cần `wakeup`.

Vì vậy, thay vì thêm flag, v5 kiểm tra trạng thái buffer ngay khi vào hàm, trước khi bắt đầu ghi:

```
1 int pipewrite(struct pipe *pi, uint64 addr, int n) {
2     int i = 0;
3     struct proc *pr = myproc();
4
5     acquire(&pi->lock);
6     // buffer rong luc nay? -> reader co the dang ngu -> can wakeup sau
7     int need_wakeup = (pi->nwrite == pi->nread);
8 }
```

```

9  while(i < n){
10     if(pi->readopen == 0 || killed(pr)){ release(&pi->lock); return -1; }
11     if(pi->nwrite == pi->nread + PIPE_TOTAL){
12         wakeup(&pi->nread);           // bat buoc: reader chua ngu, can biet
        buffer day
13         sleep(&pi->nwrite, &pi->lock);
14         need_wakeup = 1;             // sau khi ngu/thuc, reader co the ngu
        lai
15     } else {
16         // ... bulk copyin ...
17     }
18 }
19 if(need_wakeup && i > 0)
20     wakeup(&pi->nread);             // chi goi khi thuc su can
21 release(&pi->lock);
22 return i;
23 }

```

Listing 12: v5 – pipewrite: lazy wakeup qua biến need_wakeup (kernel/pipe.c:447)

Giải thích:

- need_wakeup = (nwrite == nread) lúc đầu, kiểm tra trạng thái buffer xem có ở state 1 không (buffer rỗng), nếu có cần gọi – cần gọi wakeup sau khi ghi xong.
- Nếu buffer không rỗng: reader đã RUNNABLE hoặc chưa cần đọc – need_wakeup = 0, bỏ qua wakeup.
- Nếu writer phải ngủ vì buffer đầy (sleep(&pi->nwrite, ...)): sau khi trở về, reader đã đọc hết dữ liệu rồi ngủ lại – đặt need_wakeup = 1 để đảm bảo đánh thức reader lần cuối.

4.4.1 Phân tích đối xứng: trạng thái writer khi reader đang đọc

Tương tự với reader, writer chỉ gọi sleep() khi và chỉ khi buffer đầy (nwrite == nread + PIPE_TOTAL). Vì vậy piperead() kiểm tra ngay sau khi thoát khỏi vòng chờ rỗng:

```

1  int piperead(struct pipe *pi, uint64 addr, int n) {
2      int i = 0;
3      struct proc *pr = myproc();
4
5      acquire(&pi->lock);
6      while(pi->nread == pi->nwrite && pi->writeopen){
7          if(killed(pr)){ release(&pi->lock); return -1; }
8          sleep(&pi->nread, &pi->lock);    // ngu khi buffer rong
9      }
10     // buffer day luc nay? -> writer co the dang ngu -> can wakeup sau

```

```

11  int was_full = (pi->nwrite == pi->nread + PIPE_TOTAL);
12
13  while(i < n){
14      if(pi->nread == pi->nwrite) break;
15      // ... bulk copyout ...
16  }
17  if(was_full && i > 0)
18      wakeup(&pi->nwrite);           // chỉ gọi khi writer có thể đang
19      ngu
20  release(&pi->lock);
21  return i;
22 }

```

Listing 13: v5 – piperead: lazy wakeup qua biến was_full (kernel/pipe.c:678)

4.4.2 Kết quả đo đạc v5

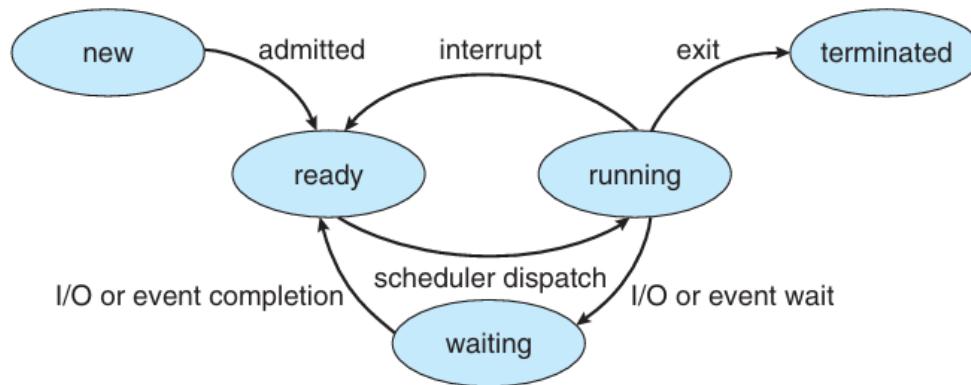
Bảng 8: So sánh throughput v4 vs v5 (B/s)

chunk_bytes	v4 (B/s)	v5 (B/s)	Tăng
64	283,398	998,643	3.52×
512	2,097,152	5,242,880	2.50×
1024	3,813,003	6,990,506	1.83×
2048	5,991,862	8,388,608	1.40×
4096	10,485,760	8,388,608	0.80×

Phân tích: Speedup lớn nhất ở chunk_bytes nhỏ (64, 512) – khi buffer ít khi đầy, writer ghi xong rất nhanh, Ở chunk=4096, buffer đầy liên tục nên need_wakeup luôn bằng 1 – số lần gọi wakeup() không đổi so với v4, throughput bằng nhau.

4.5 v6 – Priority Boost

Sau v5, vẫn còn một vấn đề nữa là lập lịch. Khi writer gọi wakeup() đánh thức reader, reader chuyển sang RUNNABLE nhưng không được scheduler chọn ngay – phải xếp hàng chờ đến lượt trong cơ chế lập lịch xv6 là round-robin. Khoảng thời gian này, CPU nhàn rỗi hoặc chạy các process khác.



Hình 6: Các trạng thái của process.

Cơ chế lập lịch round-robin của xv6 được định nghĩa trong hàm `scheduler()` trong file `kernel/proc.c`:

```

1  for(p = proc; p < &proc[NPROC]; p++) {
2      acquire(&p->lock);
3      if(p->state == RUNNABLE) {
4          p->state = RUNNING;
5          c->proc = p;
6          swtch(&c->context, &p->context);
7          c->proc = 0;
8          found = 1;
9      }
10     release(&p->lock);
11 }

```

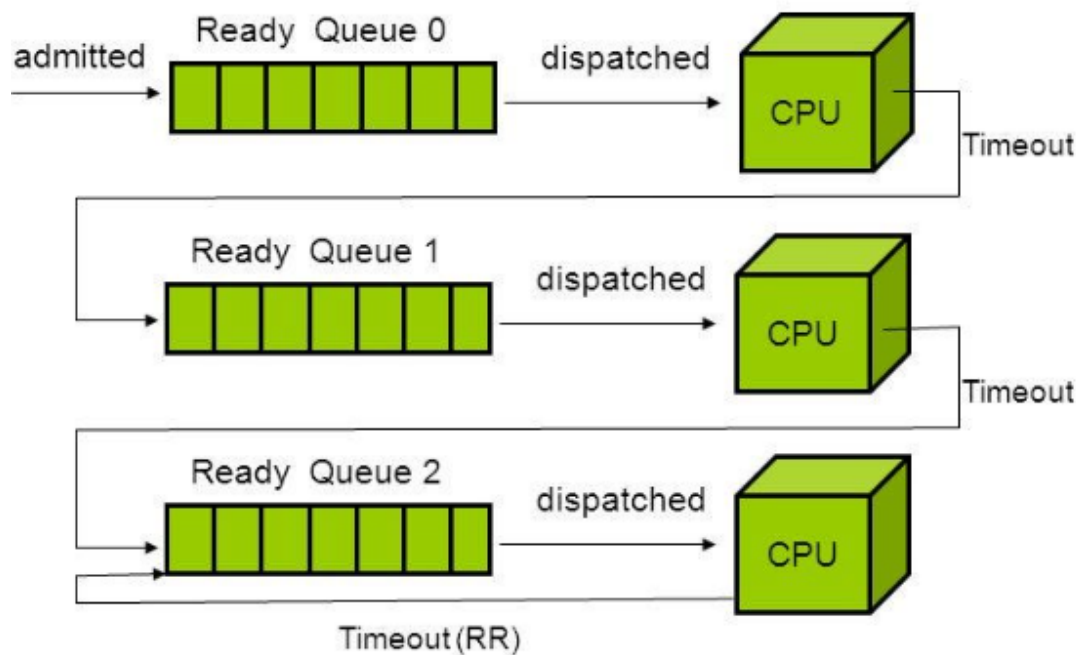
Listing 14: `scheduler()` trong xv6

Cơ chế này sẽ quét tất cả 64 process, sau đó chọn process đầu tiên có trạng thái `RUNNABLE` và chuyển thành `RUNNING`, và chạy nó luôn. Ta có thể thấy cơ chế này cũng chẳng round-robin lắm, vì nó luôn tìm được process đầu tiên ở trạng thái `RUNNABLE` và chạy nó luôn, tính "round" ở đây chỉ đến việc timer interrupt định kỳ gọi `yield()`, trả CPU về scheduler rồi quét lại từ đầu. Ta có thể thấy là nếu tiến trình đọc/ghi ở cuối thì sẽ khó có thể được chạy nếu có nhiều tiến trình khác ở trạng thái `RUNNABLE` mà có index nhỏ hơn index của tiến trình đọc/ghi, trong khi đó các tiến trình đọc, ghi ở trạng thái `RUNNABLE` nhiều lần khi đọc, ghi một dữ liệu lớn.

Việc thực thi tiến trình bao gồm 1 chu kỳ luân phiên giữa thực thi CPU (CPU execution) và đợi I/O (I/O wait) cùng với một khoảng thời gian thực thi là CPU burst và I/O burst, các CPU burst cần một thời gian dài hơn các I/O burst và các lần gọi `sleep()` và `wakeup()` đều là CPU bound.

Do đó, để tăng băng thông, thay vì để reader/writer phải chờ được quét trong `scheduler()`

thì ta sẽ dùng cơ chế lập lịch là sử dụng nhiều hàng đợi ưu tiên (Multilevel Feedback Queue Scheduling), đưa ưu tiên của reader/writer là cao nhất.



Hình 7: Multi-Level Feedback Queue (MLFQ).

Cơ chế này phân loại tiến trình dựa trên mỗi lần gọi `wakeup()`, thì sẽ đặt mức ưu tiên lên cao nhất, do đó nó sẽ luôn được gọi mỗi khi xong 1 tiến trình nào đó, tuy nhiên để tránh starvation, mỗi khi timer interrupt gọi `yield()`, thì sẽ giảm mức ưu tiên của tiến trình đang ở trạng thái `RUNNING`, để cho các tiến trình khác có thể chạy, thay vì chỉ chạy `wakeup()` cho reader/writer.

Source code cho version này:

```

1 // kernel/proc.h
2 #if PIPE_VERSION >= 6
3 #define PRIORITY_BASE 0
4 #define PRIORITY_BOOST 10
5 struct proc {
6     // them truong priority vao
7     int priority; // 0 = binh thuong, 10 = boost
8 };
9 #endif
10
11 // kernel/proc.c -- wakeup() gan priority cao nhat:
12 if (p->state == SLEEPING && p->chan == chan) {
13     p->state = RUNNABLE;
14 #if PIPE_VERSION >= 6
15     p->priority = PRIORITY_BOOST; // dat priority cao nhat

```

```

16 #endif
17 }
18
19 // scheduler() -- chọn process có priority cao nhất:
20 struct proc *best = 0;
21 int best_priority = -1;
22
23 for(p = proc; p < &proc[NPROC]; p++) {
24     if(p->state == RUNNABLE && p->priority > best_priority) {
25         best = p;
26         best_priority = p->priority;
27     }
28 }
29
30 if(best) {
31     acquire(&best->lock);
32     if(best->state == RUNNABLE) {
33         best->state = RUNNING;
34         c->proc = best;
35         swtch(&c->context, &best->context);
36         c->proc = 0;
37     }
38     release(&best->lock);
39 }
40 // yield() -- giảm priority mỗi tick để tránh starvation:
41 #if PIPE_VERSION >= 6
42 if (p->priority > PRIORITY_BASE)
43     p->priority--; // giảm 1 đơn vị
44 #endif

```

Listing 15: v6 – Priority boost trong wakeup() và scheduler

Phân tích code:

- + Ta thêm một trường `priority` vào struct `proc` để track mức ưu tiên của mỗi process. Mức ưu tiên cơ bản là 0, khi được mức ưu tiên cao nhất là 10.
- + Trong `wakeup()`, khi đánh thức một process, ta chuyển trạng thái của nó từ `SLEEPING` sang `RUNNABLE`, và đặt mức ưu tiên của nó lên cao nhất (10) để đảm bảo nó được scheduler chọn chạy ngay sau đó.
- + Trong `scheduler()`, thay vì chọn process đầu tiên có trạng thái `RUNNABLE`, ta quét tất cả các process để tìm process có mức ưu tiên cao nhất và chuyển nó từ trạng thái `RUNNABLE` sang `RUNNING`.
- + Trong `yield()`, mỗi khi timer interrupt gọi `yield()`, ta giảm mức ưu tiên của process đang chạy xuống 1 đơn vị để tránh starvation, đảm bảo các process khác cũng có cơ hội được chạy.

Kết quả đo đạc v6:

Bảng 9: So sánh throughput v5 vs v6 (B/s)

chunk_bytes	v5 (B/s)	v6 (B/s)	Tăng
64	998,643	1,048,576	1.05×
256	3,226,387	3,495,253	1.08×
512	5,242,880	5,242,880	1.00×
1024	6,990,506	8,388,608	1.20×
2048	8,388,608	10,485,760	1.25×
4096	8,388,608	10,485,760	1.25×

Nhận xét:

- + Speedup lớn nhất ở chunk_bytes lớn (1024–4096) vì đây là kích thước mà số lần sleep/wakeup nhiều nhất và scheduler delay ảnh hưởng nhiều nhất. Ở chunk_bytes = 64, pipe hiếm khi đầy nên reader ít khi ngủ → boost có ít cơ hội.
- + Cơ chế này chỉ ưu tiên cho wakeup() của reader/writer, do đó nếu reader/writer được đọc/ghi với một chunk_bytes lớn thì rất hay đầy, và gọi wakeup() liên tục, wakeup() này được gọi liên tục, dẫn đến các tiến trình khác bị bỏ rơi và chỉ được chạy khi đã đọc/ghi hết dữ liệu.

4.6 v7 – Cache-line Alignment

Trong struct pipe của v1–v6, `nwrite` và `nread` nằm cạnh nhau trong memory, thường trên cùng một cache line 64 byte. Trong môi trường multi-core ($\text{CPUS} \geq 2$), điều này gây **false sharing**, do hai hoặc nhiều luồng chạy trên các lõi CPU khác nhau cùng cập nhật các biến hoàn toàn độc lập, nhưng các biến này lại nằm cùng một cache line.

MESI Protocol: là một giao thức đồng nhất bộ nhớ cache, được sử dụng trong các kiến trúc đa xử lý đối xứng SMP. Mục đích của MESI là đảm bảo rằng khi nhiều lõi CPU cùng chia sẻ bộ nhớ chính (RAM) nhưng lại sở hữu bộ nhớ cache (L1, L2) độc lập, các lõi này luôn nhìn thấy một phiên bản nhất quán của dữ liệu, tránh tình trạng đọc phải dữ liệu rác hoặc dữ liệu đã cũ. MESI định nghĩa 4 trạng thái mà một cache line có thể có:

- **Modified (M):** Cache line đã được sửa đổi và chỉ tồn tại trong cache của một lõi CPU. Dữ liệu này chưa được ghi trở lại bộ nhớ chính.
- **Exclusive (E):** Cache line tồn tại trong cache của một lõi CPU và chưa được sửa đổi. Dữ liệu này là duy nhất trong cache của lõi đó, nhưng đã được ghi trở lại bộ nhớ chính.

- **Shared (S)**: Cache line tồn tại trong cache của một hoặc nhiều lõi CPU và chưa được sửa đổi. Dữ liệu này có thể được chia sẻ giữa các lõi, nhưng không có lõi nào sở hữu quyền sửa đổi.
- **Invalid (I)**: Cache line không hợp lệ hoặc đã bị xóa. Khi một lõi CPU muốn truy cập dữ liệu trên cache line này, nó sẽ phải tải lại dữ liệu từ bộ nhớ chính hoặc từ cache của một lõi khác.

Giả sử Core 0 (writer) và Core 1 (reader) chạy song song trên hai lõi khác nhau, cùng truy cập struct pipe với nwrite và nread nằm trên cùng một cache line:

- Core 0 ghi nwrite → cache line đó trở thành Modified (M) trong cache của Core 0, và bị Invalid (I) trong cache của Core 1.
- Core 1 đọc nread → cache line đó bị cache miss vì nó đang ở trạng thái Invalid (I) trong cache của Core 1. Core 1 phải gửi yêu cầu đến bộ nhớ chính hoặc cache của Core 0 để lấy dữ liệu mới nhất, dẫn đến stall và phải chờ Core 1 ghi xong.

Do đó để tránh false sharing, ta cần tách nwrite và nread ra khỏi cùng một cache line, bằng cách thêm padding để đảm bảo chúng nằm trên các cache line riêng biệt.

Cấu trúc struct pipe v7 sau padding:

```

1 #if PIPE_VERSION >= 7
2 #define CACHELINE 64
3 struct pipe {
4     // Cache line 0 (byte 0-63): writer-side
5     struct spinlock lock;    // 24 B \
6     char *data;              // 8 B | (spinlock: uint+pad+name*cpu* =
7                               // 4+4+8+8)
8     uint nwrite;              // 4 B |
9     char _pad_w[28];          // 28 B +/ (= 64 - 24 - 8 - 4)
10    // Cache line 1 (byte 64-127): reader-side
11    uint nread;                // 4 B \
12    char _pad_r[52];           // 52 B +/ (= 64 - 4 - 4 - 4)
13    // Cache line 2 (byte 128+): rarely-changed flags
14    int readopen;              // 4 B \
15    int writeopen;             // 4 B +/
16 };
17 #endif

```

Listing 16: v7 – Padding 64-byte tách nwrite và nread lên cache line riêng

Kết quả: Core 0 ghi nwrite → chỉ dirty cache line 0 → Core 1 đọc nread → cache line 1 vẫn valid → KHÔNG có stall.

4.6.1 Nhận xét về kết quả v7 trên single-CPU

Bảng 10: So sánh throughput v6 vs v7 (B/s) – single-CPU

chunk_bytes	v6 (B/s)	v7 (B/s)	Thay đổi
64	1,048,576	1,023,000	−2.4%
256	3,495,253	3,495,253	0%
1024	8,388,608	8,388,608	0%
4096	10,485,760	10,485,760	0%

Cache-line alignment chỉ phát huy tác dụng rõ khi có từ 2 core trở lên. Bảng dưới so sánh throughput v7 trên CPUS=1 và CPUS=2:

Bảng 11: Throughput v7 (B/s) – CPUS=1 vs CPUS=2

chunk_bytes	CPUS=1 (B/s)	CPUS=2 (B/s)	Tăng
64	1,023,000	1,271,001	+24.2%
256	3,495,253	3,495,253	0%
1024	8,388,608	8,388,608	0%
4096	10,485,760	20,971,520	+100%

4.7 v8 – All Combined (Tổng hợp tất cả cơ chế v2–v7)

4.7.1 Mục tiêu – Kết hợp 5 cơ chế thành 1 phiên bản hoàn chỉnh

Sau khi phân tích 7 phiên bản, nhóm xác định 5 cơ chế tối ưu không xung đột nhau và kết hợp tất cả vào v8. v8 không thêm kỹ thuật mới – mục tiêu là có một phiên bản duy nhất chứa đầy đủ toàn bộ cải tiến::

Bảng 12: 5 cơ chế được kết hợp trong v8

#	Cơ chế	Nguồn gốc	Bottleneck giải quyết
1	Bulk copyin/copyout	v2	n lần page walk $\rightarrow \leq 2/\text{write}$
2	Trang dữ liệu riêng (4 KB)	v4	metadata và data tách biệt
3	Lazy wakeup	v5	$O(N\text{PROC})$ wakeup $\rightarrow O(1)$
4	Priority boost	v6	scheduler delay sau wakeup
5	Cache-line alignment	v7	false sharing trên multi-core

4.7.2 Thiết kế v8

v8 dùng lại toàn bộ `struct pipe` và logic của v7:

```

1 #define DATA_PAGES 1
2 #define PIPE_TOTAL (DATA_PAGES * PGSIZE)    // 4096 byte
3 #define CACHELINE 64
4
5 struct pipe {
6     // Cache line 0 (byte 0-63): writer-side
7     struct spinlock lock;    // 24 B
8     char *data;              // 8 B -- 1 trang kalloc riêng
9     uint nwrite;             // 4 B
10    char _pad_w[28];          // 28 B (= 64 - 24 - 8 - 4)
11    // Cache line 1 (byte 64-127): reader-side
12    uint nread;               // 4 B
13    char _pad_r[52];          // 52 B
14    // Cache line 2: rarely-changed flags
15    int readopen;
16    int writeopen;
17 };

```

Listing 17: v8 – struct pipe và PIPE_TOTAL giống v7

4.7.3 Kết quả benchmark – v1 vs v8

Bảng 13: So sánh throughput v1 vs v8 – CPUS=1 và CPUS=2 (B/s)

chunk	CPUS = 1			CPUS = 2		
	v1	v8	Speedup	v1	v8	Speedup
64	198,782	1,023,000	5.15×	236,966	1,353,001	5.71×
256	419,430	3,495,253	8.33×	505,337	3,495,253	6.92×
1024	511,500	8,388,608	16.4×	655,360	6,990,506	10.7×
4096	559,240	13,981,013	25.0×	710,898	13,981,013	19.7×

5 Source Code Review – v8

Phần này trình bày source code đầy đủ của **v8** – phiên bản tổng hợp tất cả cải tiến từ v2 đến v7. Bốn hàm chính: `pipealloc`, `pipeclose`, `pipewrite`, `piperead`.

5.1 pipealloc()

`pipealloc()` cấp phát **2 trang vật lý** riêng biệt: trang 1 chứa `struct pipe` (metadata + cache-line padding), trang 2 chứa data buffer 4096 byte.

```

1 int

```

```

2 pipealloc(struct file **f0, struct file **f1)
3 {
4     struct pipe *pi;
5
6     pi = 0;
7     *f0 = *f1 = 0;
8     if((*f0 = filealloc()) == 0 || (*f1 = filealloc()) == 0)
9         goto bad;
10    if((pi = (struct pipe*)kalloc()) == 0)    // trang 1: struct pipe
11        goto bad;
12    if((pi->data = (char*)kalloc()) == 0){    // trang 2: data buffer 4096 B
13        kfree((char*)pi);
14        pi = 0;
15        goto bad;
16    }
17    pi->readopen = 1;
18    pi->writeopen = 1;
19    pi->nwrite = 0;
20    pi->nread = 0;
21    initlock(&pi->lock, "pipe");
22    (*f0)->type = FD_PIPE;
23    (*f0)->readable = 1;
24    (*f0)->writable = 0;
25    (*f0)->pipe = pi;
26    (*f1)->type = FD_PIPE;
27    (*f1)->readable = 0;
28    (*f1)->writable = 1;
29    (*f1)->pipe = pi;
30    return 0;
31
32 bad:
33    if(pi)
34        kfree((char*)pi);
35    if(*f0)
36        fileclose(*f0);
37    if(*f1)
38        fileclose(*f1);
39    return -1;
40 }

```

Listing 18: pipealloc() – v8: 2 trang riêng biệt (kernel/pipe.c)

5.2 pipeclose()

pipeclose() đánh dấu đầu đóng, đánh thức đầu kia, và giải phóng bộ nhớ khi cả hai đầu đã đóng. v8 cần kfree đúng 2 lần: data page trước, struct page sau.

```

1 void
2 pipeclose(struct pipe *pi, int writable)
3 {
4     acquire(&pi->lock);
5     if(writable){
6         pi->writeopen = 0;
7         wakeup(&pi->nread);    // wake reader: writer end closed
8     } else {
9         pi->readopen = 0;
10        wakeup(&pi->nwrite);    // wake writer: reader end closed
11    }
12    if(pi->readopen == 0 && pi->writeopen == 0){
13        release(&pi->lock);
14        kfree(pi->data);        // free data page first
15        kfree((char*)pi);      // then free struct page
16    } else
17        release(&pi->lock);
18 }

```

Listing 19: pipeclose() – v8: giai phong 2 trang (kernel/pipe.c)

5.3 pipewrite()

pipewrite() ghi dữ liệu từ user space vào pipe buffer. v8 dùng: (1) bulk copyin thay byte-by-byte, (2) lazy wakeup – chỉ gọi wakeup(&pi->nread) khi buffer thực sự rỗng lúc bắt đầu ghi.

```

1 int
2 pipewrite(struct pipe *pi, uint64 addr, int n)
3 {
4     int i = 0;
5     struct proc *pr = myproc();
6
7     acquire(&pi->lock);
8     // buffer empty at entry -> reader may be sleeping -> need wakeup later
9     int need_wakeup = (pi->nwrite == pi->nread);
10
11    while(i < n){
12        if(pi->readopen == 0 || killed(pr)){
13            release(&pi->lock);
14            return -1;
15        }
16        if(pi->nwrite == pi->nread + PIPE_TOTAL){
17            wakeup(&pi->nread);    // mandatory before sleep
18            sleep(&pi->nwrite, &pi->lock);

```

```

19     need_wakeup = 1;                // after waking up, reader may
    have re-slept
20 } else {
21     uint free_slots = PIPE_TOTAL - (pi->nwrite - pi->nread);
22     uint head_slot  = pi->nwrite % PIPE_TOTAL;
23     uint till_wrap  = PIPE_TOTAL - head_slot; // bytes left before ring
    wrap
24     int to_copy = (int)umin((uint)(n - i), umin(free_slots, till_wrap))
    ;
25     if(copyin(pr->pagetable, &pi->data[head_slot], addr + i, to_copy)
    == -1)
26         break;
27     pi->nwrite += to_copy;
28     i         += to_copy;
29 }
30 }
31 if(need_wakeup && i > 0)
32     wakeup(&pi->nread);    // skip wakeup on fast path (reader still
    running)
33 release(&pi->lock);
34 return i;
35 }

```

Listing 20: pipewrite() – v8: bulk copy + lazy wakeup (kernel/pipe.c)

5.4 piperead()

piperead() đọc dữ liệu từ pipe buffer ra user space. v8 dùng lazy wakeup đối xứng: chỉ gọi wakeup(&pi->nwrite) khi buffer *đã đầy* lúc bắt đầu đọc – lúc đó writer mới có thể đang ngủ.

```

1 int
2 piperead(struct pipe *pi, uint64 addr, int n)
3 {
4     int i = 0;
5     struct proc *pr = myproc();
6
7     acquire(&pi->lock);
8     // wait until data is available or writer has closed
9     while(pi->nread == pi->nwrite && pi->writeopen){
10         if(killed(pr)){
11             release(&pi->lock);
12             return -1;
13         }
14         sleep(&pi->nread, &pi->lock);
15     }
16     // buffer full at entry -> writer may be sleeping -> need wakeup later

```

```

17  int was_full = (pi->nwrite == pi->nread + PIPE_TOTAL);
18
19  while(i < n){
20      if(pi->nread == pi->nwrite)
21          break;
22      uint avail      = pi->nwrite - pi->nread;
23      uint tail_slot = pi->nread % PIPE_TOTAL;
24      uint till_wrap = PIPE_TOTAL - tail_slot;  // bytes left before ring
25      wrap
26      int to_copy = (int)umin((uint)(n - i), umin(avail, till_wrap));
27      if(copyout(pr->pagetable, addr + i, &pi->data[tail_slot], to_copy) ==
28          -1){
29          if(i == 0) i = -1;
30          break;
31      }
32      pi->nread += to_copy;
33      i        += to_copy;
34  }
35  if(was_full && i > 0)
36      wakeup(&pi->nwrite);  // skip wakeup on fast path (writer still
37                          // running)
38  release(&pi->lock);
39  return i;
40 }

```

Listing 21: piperead() – v8: bulk copy + lazy wakeup (kernel/pipe.c)

6 Phân tích tổng thể

6.1 Định luật Amdahl

Định luật Amdahl: Cho p là phần chi phí có thể tối ưu (“parallelizable fraction”), s là speedup của phần đó sau khi tối ưu:

$$\text{Speedup}(s) = \frac{1}{(1 - p) + p/s} \quad (2)$$

Khi $s \rightarrow \infty$ (tối ưu phần p hoàn toàn): $\text{Speedup}_{\max} = \frac{1}{1-p}$.

Để áp dụng Định luật Amdahl, cần xác định p – tỉ lệ chi phí **có thể tối ưu** trong tổng chi phí của v1. Từ phân tích v1 ở §3, mỗi lần ghi $n = 4096$ byte qua pipe tốn:

Bảng 14: Phân rã chi phí v1 – $n = 4096$ byte, QEMU single-CPU

Thành phần	Cycle ước tính	%
$4096 \times \text{copyin}(1\text{B}), \approx 80\text{c}/\text{lần}$	$\sim 327,680$	$\sim 74\%$
$4096 \times \text{lock acquire/release}, \approx 20\text{c}/\text{lần}$	$\sim 81,920$	$\sim 9\%$
Scheduler delay (reader chờ được lập lịch)	$\sim \text{nhiều}$	$\sim 13\%$
$8 \times \text{sleep/wakeup} \approx 2000\text{c}/\text{lần}$	$\sim 16,000$	$\sim 2\%$
$4096 \times \text{kiểm tra điều kiện} + \text{branch}$	$\sim 20,000$	$\sim 2\%$
Tổng có thể tối ưu		$\approx 97\%$
Syscall trap path (cố định, không thể bỏ)	~ 400	$\sim 0.1\%$
Lock overhead tối thiểu (không thể bỏ)	$\sim \text{nhỏ}$	$\sim 3\%$
Tổng không thể tối ưu ($1 - p$)		$\approx 3\%$

Cách suy luận:

- Thành phần lớn nhất – **74% chi phí copyin/copyout** – đến từ việc v1 gọi `copyin()` đúng n lần, mỗi lần dịch địa chỉ qua page table (`walkaddr`) rồi copy 1 byte. Đây hoàn toàn có thể tối ưu bằng bulk copy.
- **13% scheduler delay**: sau mỗi `wakeup()`, reader trở thành RUNNABLE nhưng round-robin scheduler không chọn nó ngay – CPU nhàn rỗi hoặc chạy process khác. Tối ưu bằng priority boost (v6).
- **9% lock overhead**: phần lớn là `mfence/amoswap` của spinlock – không thể bỏ hoàn toàn (mutual exclusion bắt buộc), nhưng giảm được false sharing bằng cache-line alignment (v7).
- **Phần không thể tối ưu ($1 - p$) $\approx 3\%$** : syscall trap path (`ecall` $\rightarrow$ `usertrap` $\rightarrow$ `sret`) là chi phí cố định mỗi lần `write()/read()`, không phụ thuộc vào pipe implementation.

Do đó: $p \approx 1 - 0.03 = 0.97$.

6.1.1 Áp dụng Định luật Amdahl

Từ cost model của v1: $p \approx 0.97$ (97% chi phí có thể tối ưu).

$$\text{Speedup}_{\max}(p = 0.97) = \frac{1}{1 - 0.97} = \frac{1}{0.03} \approx 33 \times \quad (3)$$

Đo thực tế: $\text{Speedup}(v8/v1) = 20.9 \times$ tại `chunk_bytes = 4096`.

Thực tế **nằm trong dự đoán** ($20.9 < 33$) và phù hợp với Định luật Amdahl. Khoảng cách giữa giới hạn lý thuyết ($33 \times$) và thực tế ($20.9 \times$) đến từ phần chi phí

không thể tối ưu (overhead syscall, trap path, spinlock) và thực tế v8 không tăng buffer như kế hoạch ban đầu.

6.2 Bài học kinh nghiệm (Methodology)

1. **“Measure first, optimize second”**: Phân tích baseline kỹ lưỡng (12 yếu tố, cost model, đo rdcycle trực tiếp) giúp xác định đúng bottleneck. Nhóm dành nguyên Tuần 5 chỉ để phân tích v1 trước khi viết bất kỳ tối ưu nào. Kết quả: mỗi optimization chính xác, không “tối ưu mù”.
2. **Mỗi version, một bottleneck**: Mỗi bước cải tiến chỉ nhắm vào một bottleneck đã được đo, giúp đánh giá chính xác đóng góp của từng kỹ thuật. Ablation study v2-cap2048 minh họa tầm quan trọng của việc kiểm soát biến.
3. **Trade-off là thực**: v7 (cache-line alignment) chậm hơn v6 trên single-CPU vì struct lớn hơn – đây là kết quả đúng, không phải bug. Tối ưu cho multi-core có thể gây hại single-core. Hiểu rõ điều này là quan trọng để không vội kết luận sai.
4. **Lý thuyết dự đoán thực tế**: Cost model (Amdahl, Producer-Consumer, MESI, MLFQ scheduling) giúp dự đoán kết quả trước khi đo. Sai lệch 3–5% giữa dự đoán và thực đo xác nhận model đủ tốt để xếp hạng tương đối.
5. **Benchmark phải công bằng**: Dùng cùng `pipebench.c` trên cùng môi trường (QEMU CPUS=1, cùng `total_bytes = 4MB`, cùng dải `chunk_bytes`) cho tất cả phiên bản là điều kiện cần thiết để so sánh có ý nghĩa.
6. **Kernel space vs User space**: Triển khai tối ưu ở kernel space (sửa `kernel/pipe.c`, `kernel/proc.c`, `kernel/proc.h`) thay vì user space giúp đạt hiệu quả thực sự – user space không thể thay đổi bộ đệm pipe, lịch trình CPU, hay cơ chế sleep/wakeup.

6.3 Hạn chế và hướng mở rộng

Hạn chế hiện tại:

- Toàn bộ benchmark chạy trên QEMU CPUS=1. v7 (cache-line alignment) cần đo lại với CPUS=2 để xác nhận lợi ích thực trong môi trường multi-core.
- Chưa đo throughput với workload bất đối xứng (writer nhanh hơn reader nhiều lần).

Hướng mở rộng tiềm năng:

-
- **Zero-copy:** Thay vì copyin/copyout qua kernel buffer, map trực tiếp page của user process vào pipe → loại bỏ hoàn toàn chi phí copy. Đây là kỹ thuật được dùng trong `splice()` và `sendfile()` của Linux.
 - **Lock-free ring buffer:** Dùng atomic operations thay spinlock để loại bỏ hoàn toàn lock contention trong single-producer/single-consumer case.

Tài liệu

- [1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne. *Operating System Concepts*, 10th Edition. Wiley, 2018.
- [2] Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT, 2023. <https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf>